

hmedrano-gif.github.io/CURN/tpd/Sesion_1_Vie_30_Octubre/

UNIVERSIDAD RAFAEL NÚÑEZ

Especialización en Analítica de Datos · Tecnologías de Procesamiento de Datos

Unidad 1: Fundamentos del Procesamiento Distribuido

Sesión 1 · Viernes 30 de octubre · 5:00–9:00 p. m.

Docente: Heyder Medrano

> *¿Se les ha congelado alguna vez una consulta, un script o un reporte cuando la tabla pasó de 5 o 10 millones de filas? ¿Qué hicieron?*

Avanza con las flechas ← →, con los botones de abajo o deslizando en el celular.

Su docente: Heyder Medrano Olier

Ingeniero de Sistemas · Especialista en Big Data e Inteligencia Artificial · más de 20 años entre la docencia universitaria, la infraestructura TIC y el análisis de datos

🎓 Formación

- **Ingeniero de Sistemas**
  Universidad Tecnológica de Bolívar, 2006
- **Especialista en Telecomunicaciones**
  UNAB, 2007
- **Tecnólogo en Seguridad en Redes de Computadores**
  SENA, 2010
- **Especialista Universitario en Big Data e Inteligencia Artificial**
  UNIR, España, 2024
- **Máster en Análisis y Visualización de Datos Masivos**
  UNIR, España

🏫 Docencia

- **Universidad de Cartagena**
  Facultad de Ingeniería de Sistemas, 2010–2021
- **SENA**
  instructor y administrador de Moodle, 2020 a hoy
- **Tecnológico Comfenalco**
  Minería de Datos y Visualización de Datos, 2024 a hoy
- **Universidad Sergio Arboleda y MinTIC**
  mentor del bootcamp de Análisis de Datos e IA, 2024

📊 Datos en un proyecto real

ALDESARROLLO · Proyecto Canal del Dique (2023)

- Tableros en Power BI con indicadores socioeconómicos de las comunidades.
- Procesos ETL con SQL que redujeron un 40 % el tiempo de procesamiento.
- Modelos predictivos en Python (pandas y scikit-learn) para priorizar comunidades vulnerables.

Python · SQL · Power BI · Looker Studio · Docker · Git · Azure · Redes y seguridad · Moodle

Por qué importa aquí: esta asignatura está justo entre la ingeniería (servidores, redes, contenedores) y el análisis (SQL, tableros, decisiones). Ese es el recorrido que haremos con el caso.

Tecnologías de Procesamiento de Datos

Especialización en Analítica de Datos · 3 créditos · 4 sesiones presenciales · Docente: Heyder Medrano

Sesión 1 · Vie 30 oct
5:00–9:00 p. m. · Unidad 1

Fundamentos del procesamiento distribuido: CAP, MapReduce, HDFS y YARN. **Hoy.**

Sesión 2 · Sáb 31 oct
8:00 a. m.–4:00 p. m. · Unidades 1 y 3

Taller 1A Hadoop y Spark con el mes completo; arranca el Proyecto Final; Taller 1B Kafka en tiempo real.

Sesión 3 · Vie 6 nov
5:00–9:00 p. m. · Unidad 2

La nube: AWS, Azure y Google Cloud, costos, seguridad y normativa; análisis de casos.

Sesión 4 · Sáb 7 nov
8:00 a. m.–4:00 p. m. · Unidades 2 y 3

Taller 2 en BigQuery y tablero; optimización; sustentaciones y examen práctico.

Evaluación

- **Proyecto Final**: diseño y configuración de un sistema distribuido · arranca sáb 31, se sustenta sáb 7
- **Taller práctico**: flujo de datos en tiempo real con Spark · sáb 31
- **Análisis de casos**: selección de tecnologías para un problema empresarial · vie 6, se sustenta sáb 7
- **Simulación en la nube** (Taller 2) y **examen práctico** · sáb 7

Lo que te llevas

- RA1 Configurar Hadoop o Spark para procesar un mes completo
- RA2 Seleccionar la nube adecuada con criterios de costo, seguridad y normativa
- RA3 Desarrollar un flujo en tiempo real con Kafka que no pierde eventos
- RA4 Ajustar memoria, núcleos y particiones y demostrarlo con mediciones

Un solo caso de principio a fin: 19,9 millones de viajes reales de Uber y Lyft en Nueva York. Cada sesión responde una parte de la misma pregunta de negocio.

Cómo vamos a trabajar: qué sí, qué no y por qué

En la empresa, el analista no construye la plataforma: la usa, la mide y decide con ella

Sí haremos	No haremos	Por qué
Ejecutar con el mes completo los procesos ya preparados	Instalar Hadoop, Spark o Kafka, ni montar servidores	En la empresa eso lo hacen ingeniería de datos o el proveedor de nube; el codespace ya lo trae listo
Configurar: cambiar memoria, núcleos o particiones y medir el efecto (RA1, RA4)	Configurar un clúster desde cero	Configurar, aquí, es ajustar un sistema que ya funciona y demostrar con mediciones que mejoró
Poner a andar el flujo en tiempo real con la app de viajes y comprobar que no pierde eventos (RA3)	Programar productores y consumidores desde cero	El analista necesita saber si el flujo es confiable y exacto, no escribir su código
Leer el código y explicar qué hace cada parte	Escribir o depurar programas	Basta entender cómo trabaja la herramienta para saber qué pedirle y qué esperar de ella
Consultar con SQL y armar el tablero (Spark, BigQuery, Looker Studio o Power BI)	Programar en Java o Scala	SQL y el tablero son las herramientas diarias del analista
Comparar tiempo, costo y exactitud, y recomendar una tecnología (RA2)	Memorizar comandos	Eso es lo que se evalúa: la decisión justificada con sus propias cifras

¿Y el tiempo? Las horas son las mismas. La máquina tarda lo mismo con o sin programar (cada job de Hadoop con el mes completo, entre 2,5 y 4,5 minutos). Lo que no se pierde corrigiendo comillas o sangrías se usa en interpretar y decidir.

¿Cómo se trabaja? Cada comando se copia de la guía, un recuadro a la vez. La guía muestra la salida que debe aparecer y qué hacer si algo falla.

Su papel es el de analista: lo que se evalúa es la decisión que toman con los datos, no escribir código.

Lo que necesitas para trabajar

Dónde está todo, qué se entrega y cómo cuidar el entorno

📁 Materiales

Todo está en **github.com/hmedrano-gif/CURN**: una carpeta por sesión con la presentación, la guía del taller y los PDF. La presentación se abre con el enlace ▶ de cada carpeta.

✅ Preparación Previa

Hoy, al cierre: crear tu cuenta de GitHub y verificar el correo (2 min). Mañana sábado a las 8:00 a. m., todos juntos: crear el codespace y pegar un comando hasta ver «LISTO para el Taller 1».

📓 Bitácora

En cada taller anotan sus mediciones: tiempos, memoria, núcleos, eventos por segundo. Son la evidencia para comparar tecnologías y para sustentar.

💻 Cuidar el codespace

La cuenta gratuita de GitHub trae horas limitadas al mes. Al terminar cada taller, detengan el codespace desde github.com/codespaces; encendido sigue gastando la cuota.

🎯 Proyecto Final

Diseñar y configurar un sistema distribuido que responda una pregunta del caso con el mes completo, y justificar la decisión con sus mediciones. Arranca el sábado 31 a las 11:00 y se sustenta el sábado 7.

📖 Glosario técnico

En la carpeta de la Sesión 1: HDFS, YARN, shuffle, partición y los demás términos, explicados en lenguaje sencillo. Úsenlo cuando una palabra no les resulte clara.

Todo queda en GitHub: si alguien falta o se pierde, la carpeta de la sesión tiene lo necesario para ponerse al día.

El problema: lo que necesita saber la gerencia

Viajes de Uber y Lyft en Nueva York, junio de 2025 · datos abiertos de la Comisión de Taxis y Limusinas (TLC)

19.868.009	4,04 GB	≈ 662.267	71,3 % · 28,7 %
viajes en un mes	en CSV · 25 columnas por viaje	viajes por día	Uber · Lyft

Pregunta 1 · con el mes completo (por lotes)

¿Cuántos viajes hizo cada empresa, cuánto cobró de tarifa base, cuánto pagó a sus conductores y qué parte se quedó la plataforma?

Es la pregunta con la que Parrott y Reich (2018) sustentaron ante la TLC el pago mínimo para los conductores, vigente desde 2019.

Pregunta 2 · mientras ocurre (en tiempo real)

¿En qué zonas están esperando más los pasajeros y cuánto ingresa cada empresa, minuto a minuto?

Es la señal de una tarifa dinámica: si la espera sube en una zona, se sube el precio o se mueven conductores hacia allá.

El ejercicio de Tecnologías de Procesamiento de Datos: responder estas dos preguntas con el archivo real, completo y a tiempo. Ese es el problema; las tecnologías son la forma de resolverlo.

Fuentes: TLC (2025); Parrott y Reich (2018) · 📚 Bibliografía

Por qué no se responde a tiempo, y cuánto cuesta

El negocio: US$ 556,9 millones de tarifa base en junio · US$ 127,0 millones retenidos por las plataformas (22,8 %)

Causa 1 · Volumen

Excel carga 1.048.576 filas: ve el **5,3 % del mes**.

Causa 2 · Velocidad

Unos **460 viajes por minuto**; el reporte nocturno llega tarde.

Causa 3 · Calidad

231.332 esperas negativas y 12.901 tarifas en cero o menos.

Causa 4 · Capacidad fija

Un computador no crece con los datos; comprar para el pico deja equipos ociosos.

US$ 12,2 M

en el mes, en **2.895.195 viajes** (14,6 %) en los que el pago al conductor superó la tarifa base. ¿En qué zonas y a qué horas? Solo se sabe procesando el mes completo.

US$ 49,9 M

de tarifa en **1.482.132 viajes** (7,5 %) con más de 10 minutos de espera. Los que cancelaron no aparecen en el archivo: esa pérdida solo se ve en tiempo real.

US$ 8,07 M

deja a las plataformas **LaGuardia** en un mes, la zona más rentable. Saber dónde se gana también es parte del problema.

Cifras calculadas con el archivo completo de la TLC. Plataforma = tarifa base menos pago al conductor (sin peajes, impuestos ni propinas). «En riesgo» no es «perdido»: estimar cuánto se recupera es trabajo de analista.

Cada causa tiene una tecnología que la ataca: eso es lo que vamos a medir en estas 4 sesiones.

Una noche en Nueva York: la cadena de la información

Haz clic en cada pieza: quién genera el dato, por dónde viaja, quién decide con él y cuándo lo trabajamos

Personas	**María y Carlos** pasajera y conductor	**Sofía** analista de datos	**Gerente** decide con los datos
Tecnologías de la empresa	**App móvil** genera los eventos	**Kafka** Sáb 31 recibe todo, sin perder nada	**Spark Streaming** Sáb 31 tarifa dinámica, en segundos
	HDFS + Spark / MapReduce Hoy y sáb 31 lotes nocturnos (2 a. m.)	**BigQuery** Sesión 3 almacén analítico en la nube	**Tablero** indicadores para la gerencia
Regulador y sociedad	**TLC** publica los viajes cada mes	**Parrott y Reich (2018)** analizan los ingresos	**Pago mínimo (2019)** vuelve a los conductores

Haz clic en una pieza para ver su papel en el caso.

En estas 4 sesiones seremos Sofía, la analista: cada tecnología es un eslabón de esta cadena, y el archivo de la práctica es el que publica la TLC.

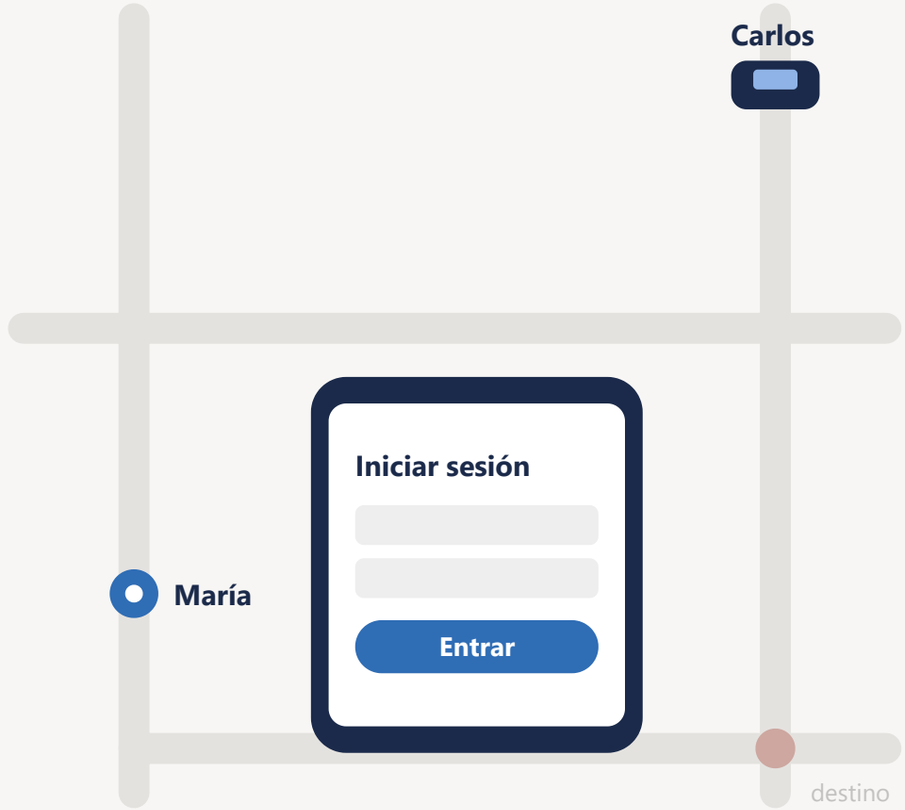
Fuentes: TLC (2025); Parrott y Reich (2018) · 📚 Bibliografía

Un viaje, de principio a fin

Desde que María entra a la app hasta que su viaje llega al tablero de la gerencia. Pausa cuando quieras o avanza etapa por etapa

1 Ingreso
2 Solicitud
3 Kafka reparte
4 Emparejamiento
5 Recogida
6 Viaje
7 Pago
8 Al tablero

Nueva York · zona ilustrativa



Kafka · tema «viajes»

esperando eventos...

Spark Streaming · en segundos

sin eventos en la ventana actual

Reparto del pago

HDFS
lote 2 a. m.

BigQuery
almacén

Tablero

Viaje ilustrativo · reparto: promedio del mes.

1. Ingreso a la plataforma. María abre la app e inicia sesión. Desde ese momento, cada acción suya es un dato.

⏸ Pausa | ← Etapa | Etapa → | ↺ Desde el inicio

Velocidad normal

El objetivo del ejercicio: de la pregunta a la tecnología

Cada herramienta entra cuando la anterior se queda corta. Haz clic en cada paso

Excel y pandas un solo computador	**Hadoop** HDFS + YARN + MapReduce	**Spark** en memoria, con SQL	**Kafka + Streaming** eventos en tiempo real	**La nube** BigQuery + tablero
Excel no abre el mes; pandas pediría unos 11 GB de memoria	Reparte el archivo en 31 bloques y el cálculo en 31 tareas	El mismo resultado, más rápido; lee directo el Parquet oficial	Responde la pregunta 2 mientras ocurren los viajes	Consulta el mes sin servidores propios y lleva la respuesta a la gerencia
Sáb 31 · Taller 1A · U1	Sáb 31 · Taller 1A · U1	Sáb 31 · Taller 1A · U1	Sáb 31 · Taller 1B · U3	Sesión 3 · Taller 2 · U2

Haz clic en un paso para ver por qué la herramienta anterior se queda corta y qué vas a medir.

Mide: tiempo, datos leídos, datos movidos y costo

Compara: el mismo resultado con otra herramienta

Decide: qué tecnología responde cada pregunta y por qué

Objetivo: decidir con evidencia qué tecnología de procesamiento responde cada necesidad del negocio, completa, exacta y a tiempo.

¿Por qué hacerlo en la nube?

El analista no compra servidores: pide capacidad cuando la necesita y la devuelve cuando termina

Simula: ¿cuántos meses de viajes quieres analizar? 1 mes

Memoria que pediría cargarlo en pandas (unos 10,8 GB por mes)



La línea roja es un computador de 16 GB.

Con el propio computador: cabe muy justo: el sistema y los programas abiertos ya usan una parte.

En la nube: se pide una máquina o un clúster con 17 GB o más, por las horas que dure el análisis, y se devuelve al terminar.

Con el propio computador

- Hay que instalar y mantener todo: Docker, Hadoop, Spark, Kafka
- Cada equipo es distinto: lo que funciona en uno falla en otro
- La capacidad es fija y no crece con los datos
- Si el equipo falla o se apaga, se detiene el trabajo

En la nube

- **Elasticidad:** 4 núcleos hoy y 400 mañana, por horas
- **Pago por uso:** BigQuery cobra por los datos que lee cada consulta
- **Servicios administrados:** el proveedor instala y opera; el analista consulta
- **Desde el navegador:** el mismo entorno para todos

Sesiones 1 y 2 · GitHub Codespaces: un computador en la nube de 4 núcleos y 16 GB, gratis y en el navegador, con Hadoop, Spark y Kafka.

Sesiones 3 y 4 · Google Cloud: BigQuery consulta los 19,9 millones de viajes sin servidores propios y sin tarjeta (Sandbox).

La nube no siempre es la respuesta: datos sensibles con restricciones legales, o cargas constantes que salen más caras alquiladas que propias. Ese análisis (seguridad, normativa y costos) es la Unidad 2.

Agenda de hoy

Hoy, la Unidad 1 — mañana sábado 31, la práctica en GitHub Codespaces

Apertura	Bloque 1	Pausa	Bloque 2	Bloque 3

5:00–5:30 · Apertura

El programa y cómo vamos a trabajar, el problema del caso, el objetivo del ejercicio, por qué la nube y las herramientas

5:30–6:30 · Procesamiento distribuido

Escalar, teorema CAP, consistencia, particionamiento y replicación

6:45–8:00 · Modelo MapReduce

Map → Shuffle → Reduce, la actividad a mano y por qué existe Spark

8:00–9:00 · Ecosistema Hadoop

HDFS, YARN, el ecosistema y los trade-offs de diseño

Hoy, el marco para evaluar arquitecturas. Mañana, sábado 31, en GitHub Codespaces: Hadoop y Spark (8:00 a. m.) y Kafka con la app de viajes (tarde).

Las herramientas de Tecnologías de Procesamiento de Datos

Qué usaremos, para qué y cuándo

Entorno de trabajo
Todas las sesiones

GitHub Codespaces
Computador en la nube (4 núcleos, 16 GB) desde el navegador

Docker
Hadoop, Spark y Kafka en contenedores

Terminal (bash)
Para dar las órdenes

Python
Datos, app de viajes y Kafka

Unidad 1 · Lotes
Sáb 31, mañana

Hadoop: HDFS + YARN
Datos repartidos en bloques y recursos del clúster

MapReduce
Procesamiento por lotes en dos fases

Spark
El mismo cálculo en memoria, para comparar

Unidad 3 · Tiempo real
Sáb 31, tarde

App de viajes
Cliente, conductor y central publican cada evento

Kafka
Mueve eventos en tiempo real

Spark Structured Streaming
Procesa eventos en ventanas de tiempo

Unidad 2 · Nube
Sesiones 3 y 4

BigQuery
SQL sin servidores; costo por bytes procesados

Looker Studio
Tablero para la gerencia

Spark en Google Cloud
Dataproc: demostración del docente

AWS y Azure
Comparativa de servicios

También existen (para ubicarse, no se practican): Databricks · Amazon EMR · Azure Synapse · Snowflake · Cloudera · Apache Flink · Amazon Kinesis · Google Pub/Sub. Los conceptos de hoy sirven para todas.

Por qué estas herramientas

Criterios de elección. Lo que se aprende con ellas sirve con cualquier proveedor

1. Lo pide el contenido programático

2. Sin costo: cuota gratuita de GitHub y BigQuery Sandbox, sin tarjeta

3. Corre igual para todos: en el navegador, sin instalar

4. Lo aprendido sirve con las alternativas

Categoría	Usamos	Alternativa principal	Por qué la nuestra
Entorno	GitHub Codespaces	Docker Desktop en el PC	4 núcleos y 16 GB en el navegador; nadie instala nada
Contenedores	Docker (en el codespace)	Podman	Imágenes oficiales de Hadoop, Spark y Kafka
Terminal	bash	PowerShell	La terminal de los servidores Linux donde corren estas plataformas
Preparar datos	Python + pandas	Excel / Power Query	El dataset supera el máximo de filas de una hoja
Lotes distribuidos	Apache Hadoop	Amazon EMR	Gratis y muestra HDFS, YARN y MapReduce por separado
Motor en memoria	Apache Spark	Apache Flink	Un solo motor para lotes (U1) y tiempo real (U3)
Nube	Google BigQuery	Amazon Redshift	Muestra el costo de cada consulta; Sandbox sin tarjeta
Tiempo real	Apache Kafka	Amazon Kinesis	Gratis; sus conceptos se usan en los servicios de nube

BLOQUE 1 · 5:30–6:30 p. m.

Procesamiento distribuido y arquitecturas

> *¿Qué pasa cuando una base de datos ya no cabe en una sola máquina?*

Bloque 1 · Procesamiento distribuido | Bloque 2 · Modelo MapReduce | Bloque 3 · Ecosistema Hadoop

¿Qué pasa cuando una sola máquina no alcanza?

Dos caminos: una máquina más grande o más máquinas trabajando juntas. Mueve el volumen de datos

Volumen del cierre nocturno: ×1

Escalar verticalmente

Una sola máquina cada vez más grande.



Línea roja: el techo físico de un servidor.

Alcanza: un servidor más grande procesa el cierre (pero cada salto cuesta más).

Escalar horizontalmente

Más máquinas comunes trabajando juntas.



1 nodo procesa el cierre. Si uno falla, los demás siguen; el costo es coordinarlos.

Camino vertical: alcanza hasta el próximo techo y sigue siendo un único punto de falla.

Camino horizontal: se crece agregando nodos, pero hay que coordinarlos.

Escalar horizontalmente exige coordinación: esa coordinación es exactamente lo que estudia el procesamiento distribuido.

Fuentes: Kleppmann (2017), cap. 1 · 📚 Bibliografía

Teorema CAP

Si la red se parte, un sistema distribuido debe elegir: consistencia o disponibilidad

C · Consistencia

Toda lectura devuelve la escritura más reciente, sin importar qué nodo responda.

A · Disponibilidad

Todo nodo que funciona responde siempre, aunque otros fallen.

P · Tolerancia a particiones

El sistema sigue operando aunque se pierdan mensajes entre nodos.

Escenario: el nodo 1 registró un retiro de US$ 200; la red se parte y el nodo 2 no lo recibió. Un cliente consulta su saldo en el nodo 2.



1. Partir la red

Responder como sistema CP

Responder como sistema AP

En un sistema distribuido real las particiones ocurren: P no se elige. La decisión es entre C y A, y es de negocio.

Fuentes: Brewer (2000); Gilbert y Lynch (2002); Brewer (2012) · Bibliografía

Consistencia fuerte vs. consistencia eventual

Qué ve quien lee justo después de una escritura, cuando el dato tiene tres réplicas

▶ Escribir con consistencia fuerte | ▶ Escribir con consistencia eventual

Nodo 1	Nodo 2	Nodo 3
100	**100**	**100**

Escribe «likes = 101» en el Nodo 1 y mira qué lee alguien en el Nodo 3.

Contador de «likes»: ver 100 en vez de 101 un instante no le cuesta nada a nadie: eventual basta.

Saldo bancario: leer un saldo anterior al retiro permite gastar dos veces el mismo dinero: exige fuerte.

La elección no es solo técnica: depende del costo de negocio de un dato desactualizado.

Fuentes: Vogels (2009); Kleppmann (2017), cap. 5 · Bibliografía

Tolerancia a fallos y particionamiento

Los viajes repartidos por distrito en 3 nodos. Haz clic en un nodo para apagarlo

☑ Con réplicas

Nodo 1	Nodo 2	Nodo 3
P1 · Manhattan	P2 · Brooklyn	P3 · Queens
réplica P3 · Queens	réplica P1 · Manhattan	réplica P2 · Brooklyn

> Todos los nodos funcionan: se pueden consultar los viajes de Manhattan, Brooklyn y Queens.

- **Particionamiento (sharding):** dividir los datos entre nodos para repartir la carga.
- **Replicación:** guardar copias del mismo dato en varios nodos para sobrevivir a fallos.

Todo diseño distribuido combina ambos, y cada combinación implica un trade-off distinto de costo, velocidad y riesgo.

Fuentes: Kleppmann (2017), caps. 5 y 6 · Bibliografía

BLOQUE 2 · 6:45–8:00 p. m.

Modelo MapReduce

> *¿Qué pasos de un problema de su trabajo serían «map» y cuáles «reduce»?*

Bloque 1 · Procesamiento distribuido | **Bloque 2 · Modelo MapReduce** | Bloque 3 · Ecosistema Hadoop

Map → Shuffle → Reduce

El job del taller de mañana: tarifa base por empresa (tarifas ilustrativas, en dólares). Avanza fase por fase

▶ Siguiente fase | ↺ Reiniciar | Entrada

Entrada (bloques)

- Bloque 1: Uber 73 · Lyft 24
- Bloque 2: Uber 64 · Uber 31
- Bloque 3: Lyft 18 · Uber 12

Map (pares clave, valor)

…

Shuffle (agrupa por clave)

…

Reduce → Salida

…

Map: cada nodo procesa su porción de datos de forma independiente y en paralelo.

Shuffle: los resultados intermedios se agrupan por clave y viajan por la red.

Reduce: cada nodo agrega los valores de una clave y produce el resultado final.

El shuffle es la fase más costosa: mueve datos por la red entre todos los nodos.

Fuentes: Dean y Ghemawat (2008) · Bibliografía

Actividad: MapReduce a mano

15 minutos · 12 viajes reales del archivo de la TLC · grupos de estudiantes hacen de mappers y de reducers

1. Map | 2. Shuffle | 3. Reduce | 4. Repetir con combiner | ↺ Reiniciar

Bloque 1 · mapper 1

Uber US$ 73,25 | Uber US$ 64,49 | Lyft US$ 22,44 | Uber US$ 32,99

Bloque 2 · mapper 2

Uber US$ 57,12 | Uber US$ 19,54 | Lyft US$ 17,90 | Uber US$ 8,84

Bloque 3 · mapper 3

Uber US$ 8,38 | Uber US$ 28,00 | Lyft US$ 6,40 | Lyft US$ 12,71

Reducer Uber

Reducer Lyft

Pulsa «1. Map»: cada mapper escribe un papel por viaje con (empresa, tarifa).

Sin combiner viajan 12 papeles; con combiner, 6 (dos por mapper). Con el mes completo: 19,9 millones de pares frente a 62.

Fuentes: Dean y Ghemawat (2008); datos: TLC (2025) · 📚 Bibliografía

Las limitaciones de MapReduce, no solo sus fortalezas

Un algoritmo iterativo repite el mismo cálculo varias veces sobre los mismos datos. Elige cuántas pasadas

Pasadas sobre los viajes: 10



MapReduce: cada pasada es un job nuevo

disco → job 1 → disco | disco → job 2 → disco | disco → job 3 → disco | disco → job 4 → disco | disco → job 5 → disco | disco → job 6 → disco | disco → job 7 → disco | disco → job 8 → disco | disco → job 9 → disco | disco → job 10 → disco

10 jobs: 10 lecturas y 10 escrituras a disco.

Spark: los datos quedan en memoria

disco | memoria: 10 pasadas | disco

1 lectura, 10 pasadas en memoria y 1 escritura al final.

- **Disco entre jobs:** cada job escribe los resultados intermedios a disco.
- **Iterar es relanzar:** los algoritmos iterativos (por ejemplo, de machine learning) relanzan el job en cada pasada.

Esta limitación concreta es la razón de ingeniería detrás de Apache Spark. Mañana la comprueban con métricas.

Fuentes: Zaharia et al. (2010); Zaharia et al. (2012) · Bibliografía

BLOQUE 3 · 8:00–9:00 p. m.

Ecosistema Apache Hadoop

> *¿Por qué HDFS replica cada bloque 3 veces por defecto?*

Bloque 1 · Procesamiento distribuido | Bloque 2 · Modelo MapReduce | **Bloque 3 · Ecosistema Hadoop**

¿Qué es cada tecnología?

Las tres plataformas de la asignatura, en una frase cada una, y qué hacen en nuestro caso

Apache Hadoop

Plataforma de código abierto para **guardar y procesar grandes volúmenes de datos repartiéndolos entre varios computadores**. Tiene tres piezas:

- **HDFS:** el sistema de archivos distribuido; parte cada archivo en bloques y guarda copias en varias máquinas.
- **YARN:** el gestor de recursos; reparte la memoria y los núcleos del clúster entre las tareas.
- **MapReduce:** el modelo de procesamiento por lotes, en dos fases: map y reduce.

En el caso: el mes completo de viajes, por lotes (pregunta 1).

Apache Spark

Motor de **procesamiento distribuido que trabaja en memoria**. Con una misma herramienta hace lotes, consultas SQL, flujos en tiempo real (Structured Streaming) y aprendizaje automático (MLlib).

Puede correr sobre YARN y leer de HDFS: no reemplaza a Hadoop como almacén, reemplaza a MapReduce como motor.

En el caso: el mismo cálculo del mes, más rápido, y la espera por zona minuto a minuto.

Apache Kafka

Plataforma distribuida de **transmisión de eventos**: recibe los eventos a medida que ocurren, los guarda en orden y los entrega a todos los sistemas que los necesitan (publicación/suscripción).

Guarda copias en varios servidores para no perder ninguno, y un sistema que se cae retoma donde quedó.

En el caso: cada paso del viaje (solicitud, recogida, pago) como un evento en tiempo real (pregunta 2).

Hadoop guarda y procesa por lotes; Spark procesa rápido, en lotes o en flujo; Kafka mueve los eventos en tiempo real. Juntas forman la cadena del caso.

Fuentes: The Apache Software Foundation (s. f.), documentación de Hadoop, Spark y Kafka; White (2015); Zaharia et al. (2012); Kreps et al. (2011) · Bibliografía

HDFS: Hadoop Distributed File System

Un archivo se divide en bloques y cada bloque se guarda 3 veces en máquinas distintas. Haz clic en un DataNode para apagarlo

NameNode *metadatos: dónde está cada bloque*

Apagar el NameNode

DataNode 1	DataNode 2	DataNode 3	DataNode 4
B1 B2	B1 B3	B2 B3	B1 B2 B3

Los 3 bloques del archivo (B1, B2, B3) están disponibles, cada uno en varias máquinas.

En el taller de mañana: el mes (4,04 GB) queda en 31 bloques de 128 MB. Allí hay un solo DataNode, así que la réplica es 1.

El NameNode único es una decisión de diseño con una consecuencia clara: sin alta disponibilidad, es un punto único de falla.

Fuentes: Ghemawat et al. (2003); Shvachko et al. (2010); Apache Software Foundation, HDFS architecture · Bibliografía

YARN: gestión de recursos y scheduling

YARN reparte la memoria del clúster en contenedores. Elige cuánta memoria tiene el nodo (como en el paso 7 del taller)

4.096 MB | 6.144 MB

ApplicationMaster · 2.048 MB | tarea map 1 · 1.024 MB | tarea map 2 · 1.024 MB

Con 4.096 MB caben 2 tareas map a la vez. Las 31 tareas del mes van por tandas de 2 (16 tandas).

- **ResourceManager:** decide qué aplicación recibe qué recursos.
- **NodeManager:** ejecuta las tareas en cada nodo.
- **ApplicationMaster:** uno por aplicación; pide contenedores y coordina sus tareas.

Pregunta para mañana: si con más memoria caben más tareas a la vez, ¿el job siempre termina antes? En el codespace hay 4 núcleos: lo van a medir.

YARN separa los recursos del modelo de procesamiento: por eso Spark puede correr sobre la misma infraestructura que Hadoop.

Fuentes: Vavilapalli et al. (2013); Apache Software Foundation, Apache Hadoop YARN · Bibliografía

El ecosistema sobre HDFS y YARN

Informativo: herramientas que se apoyan en las capas vistas hoy

MapReduce
Lotes; escribe a disco entre fases

Hive
SQL por lotes sobre HDFS

Spark
Spark SQL · MLlib · Structured Streaming

Impala
SQL interactivo con procesos propios

YARN — gestión de recursos

HDFS — almacenamiento distribuido

Idea para llevarse: el mismo HDFS sirve a motores distintos; elegir motor es elegir entre latencia, tipo de trabajo (lotes, interactivo, iterativo) y costo.

Fuentes: Thusoo et al. (2009); Kornacker et al. (2015); Zaharia et al. (2012) · 📚 Bibliografía

Trade-offs de diseño: no hay arquitectura gratuita

Cada decisión de Hadoop compra algo y lo paga con otra cosa

Lo que se gana	Decisión de Hadoop	Lo que se paga
Mayor tolerancia a fallos	**Más réplicas**	Mayor costo de almacenamiento
Arquitectura simple	**NameNode único**	Punto único de falla
Tolerancia a fallos robusta	**Escritura a disco entre fases**	Menor velocidad

Cada decisión de Hadoop es defendible, y cada una tiene un costo que hay que poder justificar.

Fuentes: White (2015); Kleppmann (2017) · Bibliografía

Cierre del día

Hoy

Escalar | Teorema CAP | Consistencia | Particionamiento y replicación | MapReduce | HDFS | YARN | Trade-offs

Mañana, sábado 31 de octubre, 8:00 a. m.

1. Preparación Previa todos juntos: crear el codespace y ver «LISTO»

2. Ejecutar y perfilar un job MapReduce real con el mes completo

3. Hacer lo mismo con Spark y comparar

4. Arranca el Proyecto Final; en la tarde, Kafka y la app de viajes

Antes de irse: creen su cuenta de GitHub en github.com/signup y verifiquen el correo. Mañana a las 8:00 a. m. creamos el codespace todos juntos. Armen hoy sus grupos de 3 del Proyecto Final.

Bibliografía

Las fuentes de cada concepto de la sesión (normas APA, 7.ª ed.). Cada enlace abre la fuente en una pestaña nueva

El caso

New York City Taxi and Limousine Commission. (2025). *TLC trip record data: High volume for-hire vehicle trip records, junio de 2025* [Conjunto de datos]. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Parrott, J. A., & Reich, M. (2018). *An earnings standard for New York City's app-based drivers: Economic analysis and policy assessment*. Informe para la New York City Taxi and Limousine Commission. Center on Wage and Employment Dynamics, UC Berkeley. https://irle.berkeley.edu/publications/irle-policy-brief/an-earnings-standard-for-new-york-citys-app-based-drivers/

Bloque 1 · Procesamiento distribuido

Brewer, E. A. (2000). Towards robust distributed systems (resumen). En *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing* (p. 7). ACM. https://doi.org/10.1145/343477.343502

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News, 33*(2), 51–59. https://doi.org/10.1145/564585.564601

Brewer, E. (2012). CAP twelve years later: How the "rules" have changed. *Computer, 45*(2), 23–29. https://doi.org/10.1109/MC.2012.37

Vogels, W. (2009). Eventually consistent. *Communications of the ACM, 52*(1), 40–44. https://doi.org/10.1145/1435417.1435432

Bloque 2 · MapReduce y Spark

Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM, 51*(1), 107–113. https://doi.org/10.1145/1327452.1327492

Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster computing with working sets. En *2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 10)*. USENIX. https://www.usenix.org/conference/hotcloud-10/spark-cluster-computing-working-sets

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. En *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)* (pp. 15–28). USENIX. https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia

Bloque 3 · Ecosistema Hadoop

Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). The Google file system. En *Proceedings of the 19th ACM Symposium on Operating Systems Principles* (pp. 29–43). ACM. https://doi.org/10.1145/945445.945450

Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop distributed file system. En *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)* (pp. 1–10). IEEE. https://doi.org/10.1109/MSST.2010.5496972

Vavilapalli, V. K., Murthy, A. C., Douglas, C., Agarwal, S., Konar, M., Evans, R., Graves, T., Lowe, J., Shah, H., Seth, S., Saha, B., Curino, C., O'Malley, O., Radia, S., Reed, B., & Baldeschwieler, E. (2013). Apache Hadoop YARN: Yet another resource negotiator. En *Proceedings of the 4th Annual Symposium on Cloud Computing* (pp. 1–16). ACM. https://doi.org/10.1145/2523616.2523633

Thusoo, A., Sarma, J. S., Jain, N., Shao, Z., Chakka, P., Anthony, S., Liu, H., Wyckoff, P., & Murthy, R. (2009). Hive: A warehousing solution over a map-reduce framework. *Proceedings of the VLDB Endowment, 2*(2), 1626–1629. https://doi.org/10.14778/1687553.1687609

Kornacker, M., Behm, A., Bittorf, V., Bobrovytsky, T., Ching, C., Choi, A., Erickson, J., Grund, M., Hecht, D., Jacobs, M., Joshi, I., Kuff, L., Kumar, D., Leblang, A., Li, N., Pandis, I., Robinson, H., Rorke, D., Rus, S., … Yoder, M. (2015). Impala: A modern, open-source SQL engine for Hadoop. En *7th Biennial Conference on Innovative Data Systems Research (CIDR 2015)*. https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper28.pdf

The Apache Software Foundation. (s. f.). *HDFS architecture*. Apache Hadoop. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

The Apache Software Foundation. (s. f.). *Apache Hadoop YARN*. Apache Hadoop. https://hadoop.apache.org/docs/stable/hadoop-yarn/hadoop-yarn-site/YARN.html

Libros de consulta

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media. ISBN 978-1-4493-7332-0. https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/

White, T. (2015). *Hadoop: The definitive guide* (4.ª ed.). O'Reilly Media. ISBN 978-1-4919-0163-2. https://www.oreilly.com/library/view/hadoop-the-definitive/9781491901687/